Version control with git

or how to write code without going crazy

Fynn Freyer (Content by Prof. Dr.-Ing. Dabrowski)

CQ Bildung/ABI-2025-3

January 15, 2026

Agenda – Day One

- Introductions
  - Who am I?
  - Who are you?
- Overview
  - Short history of VCS
  - Motivation
  - Basic Commands
- Working with git
  - GitHub
  - Lots of practice!

Introductions

Introductions

Who am I?

- Studied Applied Computer Science at HTW-Berlin
- Specialized in Healthcare Computing
- Bioinformatics projects with Robert Koch Institute
  - Data preparation and analysis for NGS data
  - HIV Surveillance

Introductions

Who are you?

- Introduce yourselves – two minutes
- Two questions
  - “Who are you?”
    - Name, etc.
    - Professional Background
  - “What are you aiming to achieve with the knowledge from this course?” – Deep or shallow, up to you
    - Make money?
    - Fight cancer, bc. Mom died from it?

Overview

Overview

History of Version Control Systems

- Aim: Not losing control!
  - 1972: SCCS, single user
- Bonus: Coordination of work
  - 1986: CVS (central)
  - 2005: git (distributed)
  - Other systems: SVN, TFVC, BitKeeper, Mercurial etc.
- For all **text based** content (code, LaTeX, Markdown etc.)

Overview

Motivation

- Things change
  - we understand the problem better
  - requirements are updated
  - we find edge cases/bugs
  - etc.
- Our code changes accordingly
- We need to keep track of these changes

Figure 1: Revisions

Overview

Motivation

- Course diary
  - General notes
  - Cheatsheets
  - Link collection
  - Code for the exercises
- No chaos!
- Requirements may change over time

Overview

Basic Commands

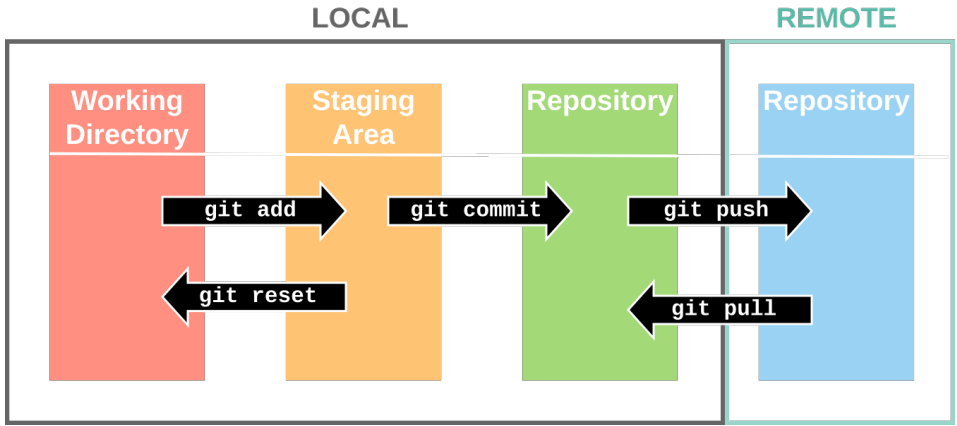

Figure 2: Basic operations

Overview

Files Over Time

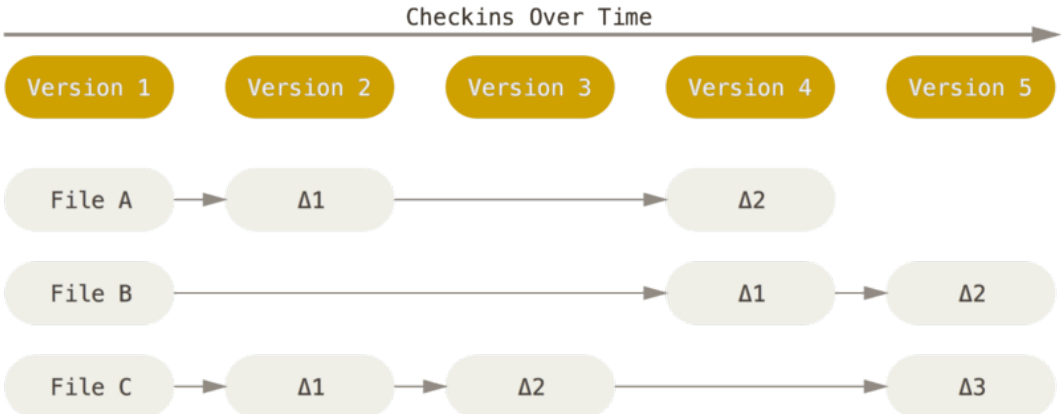

Figure 3: Git only saves changes

Overview

Repositories

- What’s a repository?

Overview

Repositories

- Where our code lives
- “A folder” (with a little extra)

> *a place, room, or container where something is deposited or stored*
> – *Merriam Webster*

Overview

Repository Providers

- GitHub
  - Many extensions (Zenodo integration, CITATION.cff)
  - Limited number of private repositories
  - Owned by Microsoft
- GitLab
  - Unlimited number of private repositories
  - Can be self-hosted
  - Not owned by Microsoft
- Many more…
- Provider not necessarily needed…

Overview

Repository Settings

- Public vs. private repositories
  - When should it be public vs. private?
  - When should a private repo be made public?
- Together:
  - Creating a repository
  - Permission management
- Short discussion: PW reuse, PW manager

Working with git

Working with git

Exercise: Creating a repository

- Create account at GitHub[1]
- Create a public[2] repository
- Invite me

[1] We use GitHub, because we need CITATION.cff and Zenodo later.

[2] Necessary to use insights.

Working with git

Access

- Do one of the following
- Add an SSH key
  - type: authentication
- Create a personal access token
  - scope: repo
  - expiration: up to you

Working with git

Important commands

- First, clone the repo:
  - `git clone user@host:path/to/repo.git` (**SSH**) or
  - `git clone https://host/path/to/repo.git` (**HTTPS**)
- Typical workflow afterwards:
  - `git add file1 file2 ...`
  - `git status`
  - `git commit -m "did the thing!"`
  - `git push`

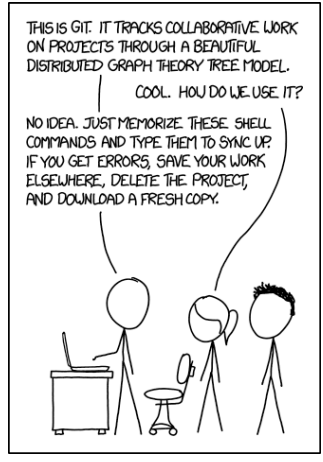

Figure 4: XKCD #1597 – A typical git user

Working with git

References

- Overview: NeSI reference sheet
- In case of fire: Oh Shit, Git!?!

Network Graph

Figure 5: Fresh repository

Exercise: Commit & Push

- Create a new text file: git cheat sheet
  - Write down the four basic commands: `add`, `status`, `commit`, and `push`
  - Two lines per command: `command`, `short description`
- Track your work using git
  - add the file to the staging area
  - check the status
  - commit your changes
- Sync to remote
  - look at repo & graph online: What happened?
  - push your commit
  - look at repo & graph online: Difference?

Working with git

Network Graph

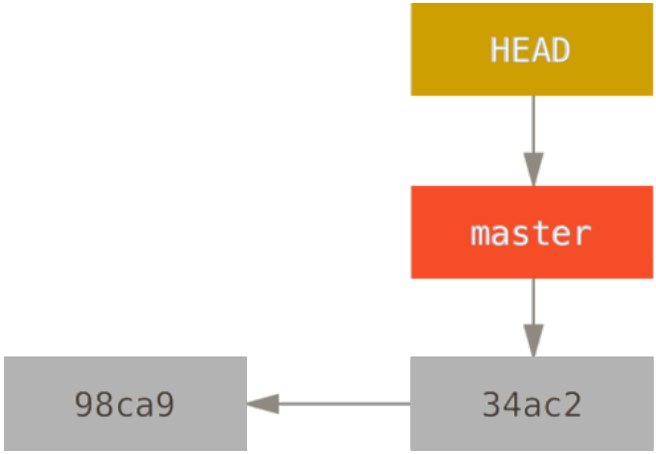

Figure 6: Repository after one commit

Markdown

- Plaintext for cheat sheet etc. is not pretty
- Markdown: popular text-based format with many features
  - Headings, bold, italic…
  - Lists, tables, images
  - Syntax highlighting for many languages, blockquotes
- Used for notes in many environments, e.g.,
  - GitHub/GitLab
  - Jupyter Notebooks
  - Reddit, Slack, Stack Overflow, Discord, Trello, and many more…
- Large Ecosystem of apps and extensions
  - Extension for slides: marp
  - Editors: dillinger.io, Joplin, Obsidian, VS Code, etc.

Working with git

Exercise: Second Commit

- Move git cheat sheet to markdown:
  - Rename to .md (with `git mv`!)
  - status, commit, push
  - Title: “Basic git commands”
  - Subtitle for each command, comment underneath
  - Add `git mv`
- add, status, commit, push

Working with git

Network Graph

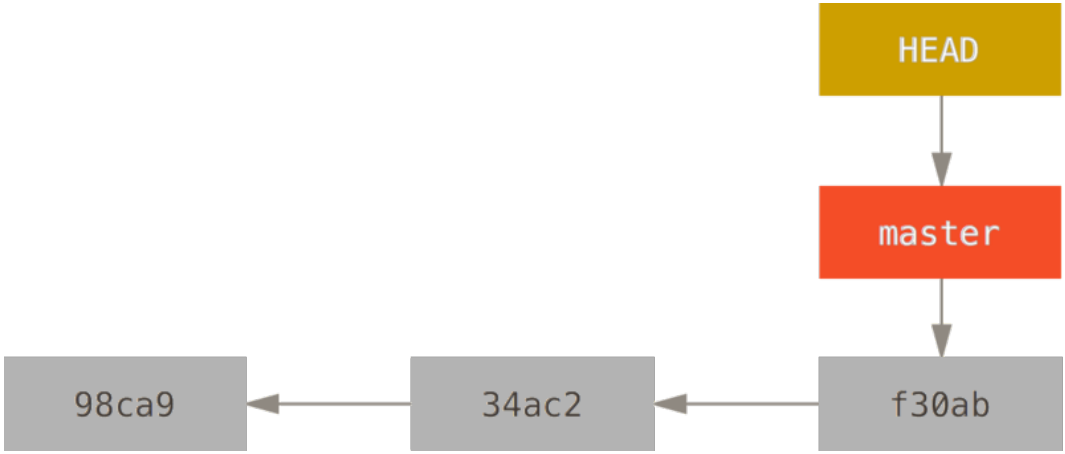

Figure 7: Repository after two commits

Working with git

Exercise: Making a mistake

- “Mistakenly” remove one of the lines from the markdown file
- add, status, commit, push

Working with git

Webinterface and history

- History view online:
  - List of commits
  - What happened in a commit
- Recover previous versions:
  - `git log`
  - Optionally `git diff --name-only <hash>`
  - `git diff <hash> -- <filename>`
  - `git checkout <hash>`

Working with git

Detached head?

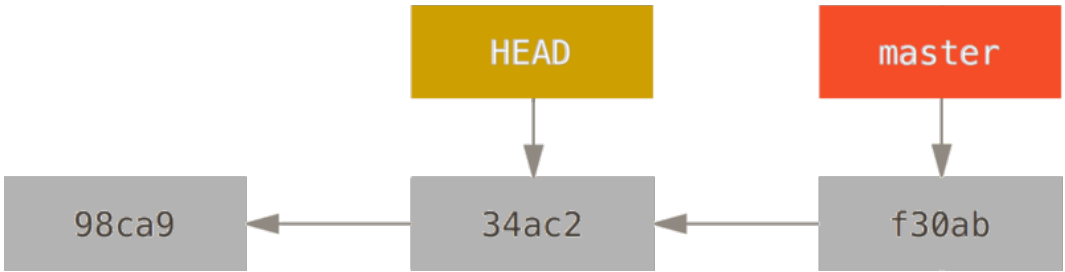

Figure 8: A repository with a detached HEAD

Working with git

Exercise: Handling mistakes

- Checkout commit with text file, verify content
- Remember commit hash
- Checkout master
- `git checkout <commit hash> -- <correct file>`
- add, commit, push

Pandoc

- One advantage of Markdown: Conversion to other formats
- Typically: `pandoc -o file.pdf file.md`
- Formattig hints and metadata can be put in file header:

```
---
title: Git Cheat Sheet
author: John Doe
geometry: margin=25mm
colorlinks: true
---
```

Working with git

Exercise: Pandoc

- build the PDF
- add, commit, push

> **Info**
>
> Usually you **shouldn't** put generated files under version control, as long as the sources are version controlled. This is just for the purposes of the exercise.

Keeping it Clean

- But does the PDF belong in the repository?
- Files to ignore can be listed in `.gitignore`
- Documentation in git-scm
- Generally: Patterns with `*`, comments with `#`, negation with `!`

```
# Ignore all temporary editor files (starting with a dot)
.*
# Ignore all files in the tmp subdirectory...
/tmp/*
# ...except temp.config
!/tmp/temp.config
```

Exercise: .gitignore

- `git rm <filename.pdf>`
- commit, push
- Build the pdf again
- Create `.gitignore`
- `git status` - what files are listed?
- Add pattern for pdf files to `.gitignore`
- `git status`: Compare to before
- add, commit, push

Folie 6: “notFinal.doc”, Jorge Cham, “Piled Higher and Deeper”, PHD comics
Folie 8: “Git: Reference Sheet”, New Zealand eScience Infrastructure
Folie 9: “Deltas”, Scott Chacon und Ben Straub, Pro Git book
Folie 16: “Git”, Randall Munroe, XKCD Folien 17, 19, 22, 25, 32-37: “Git Branching”, Scott Chacon und Ben Straub, Pro Git book